



第四章 数据库安全性



■ 问题的提出

- 数据库的一大特点是数据可以共享
- 数据共享必然带来数据库的安全性问题
- 数据库系统中的数据共享不能是无条件的共享

例： 军事秘密、国家机密、新产品实验数据、
市场需求分析、市场营销策略、销售计划、
客户档案、医疗档案、银行储蓄数据



数据库安全性

- 
- ❖数据库的安全性问题和计算机系统的安全性**紧密联系**
 - ❖计算机系统的安全性问题可分技术安全类、管理安全类和政策法律类三大类安全性问题。
 - ❖本章讨论数据库技术安全类问题，即从技术上如何保证数据库系统的安全性



数据库安全性：

数据库的安全性是指保护数据库系统和数据，以**防止不合法使用**所造成的数据泄露、更改或破坏。

数据成为一个部门、企业、乃至一个国家的重要资源，**系统安全保护措施**是否有效是数据库系统主要的性能指标之一。

- 
- 对数据库安全性产生威胁的因素主要有：
 - 非授权用户对数据库的恶意存取和破坏
 - 数据库中重要或敏感的数据被泄漏
 - 安全环境的脆弱性



讨论

问题：常常听到"**SQL注入**"，是怎么回事？

- 这是一种借助无需注册的查询界面实施的一种数据库入侵行为，利用SQL语句拼装和连续执行SQL语句机制，非法执行SQL语句窃取用户信息。如果防护不当，容易造成严重损失。
- 输入20250001，输出：SELECT * from Student wheresno="20250001";
- 输入20250001;SELECT*From Student，输出：SELECT* from Student where sno= "20250001; SELECT* From Student";



4.0 安全标准简介（了解）

- 1985年美国国防部（DoD）正式颁布《DoD可信计算机系统评估准则》（简称TCSEC或DoD85）
- 1991年，DoD颁布TCSEC/**TDI**，把TCSEC扩展到数据库系统（TDI - 可信数据库解释）
- 1993年起多个国家组织专门委员会开发了IT安全通用准则（Common Criteria, CC）
- 1999年 CC V2.1版被ISO采用为国际标准
2001年 CC V2.1版被我国采用为国家标准



TCSEC/TDI安全级别划分

- 4组 (division) 7个等级
 - D
 - C (C1, C2(MySQL))
 - B (B1, B2, B3)
 - A (A1)
- 系统可靠或可信程度逐渐增高



■ TCSEC/TDI安全级别划分

安全级别	定 义
A1	验证设计 (verified design)
B3	安全域 (security domains)
B2	结构化保护 (structural protection)
B1	标记安全保护 (labeled security protection)
C2	受控的存取保护 (controlled access protection)
C1	自主安全保护 (discretionary security protection)
D	最小保护 (minimal protection)



■ CC

- 1999年被ISO采用为国际标准
- 根据系统对安全保证要求的支持情况提出了评估保证级 (Evaluation Assurance Level, EAL)
 - 从EAL1至EAL7共分为7级
 - 按保证程度逐渐增高



评估保证级	定 义	TCSEC安全级别 (近似相当)
EAL1	功能测试 (functionally tested)	
EAL2	结构测试 (structurally tested)	C1
EAL3	系统地测试和检查 (methodically tested and checked)	C2
EAL4	系统地设计、测试和复查 (methodically designed, tested, and reviewed)	B1
EAL5	半形式化设计和测试 (semiformally designed and tested)	B2
EAL6	半形式化验证的设计和测试 (semiformally verified design and tested)	B3
EAL7	形式化验证的设计和测试 (formally verified design and tested)	A1

CC评估保证级 (EAL) 的划分及与TCSEC/TDI安全级别的对应



第4章 数据库安全性

4.1 数据库安全性控制

4.2 视图机制

4.3 审计

4.4 数据加密

4.5 其他安全性保护（自学）



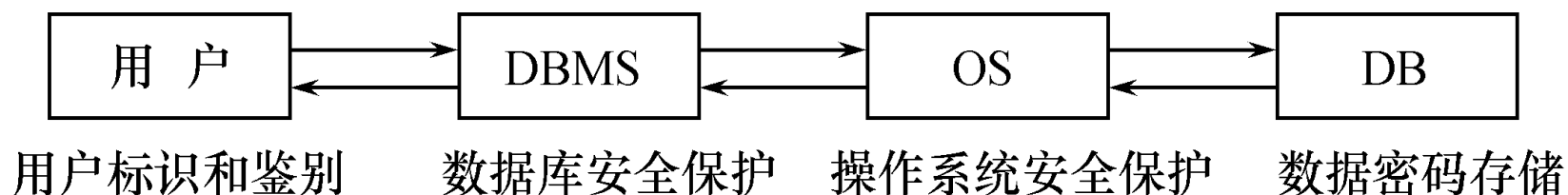
4.1 数据库安全性控制

4.1.1 用户身份鉴别

4.1.2 存取控制

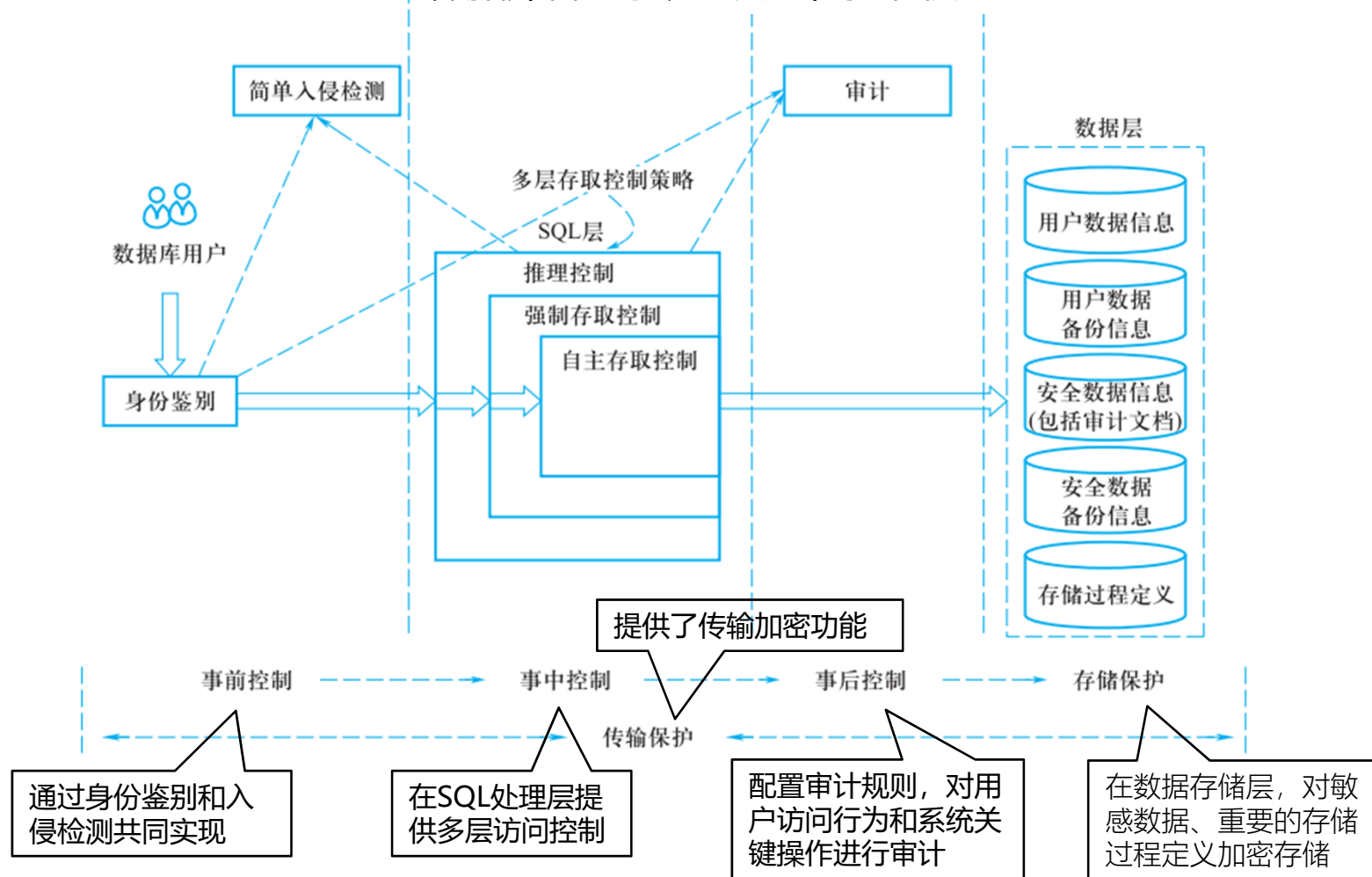
4.1.3 数据库角色

- 计算机系统中，安全措施是一级一级层层设置



计算机系统的安全模型

数据库管理系统的安全性控制模型





❖ 数据库管理系统的安全性控制模型

- ①事前控制：通过身份鉴别和入侵检测共同实现
- ②事中控制：在SQL处理层提供多层访问控制
- ③事后控制：配置审计规则，对用户访问行为和系统关键操作进行审计
- ④存储保护：在数据存储层，对敏感数据、重要的存储过程定义加密存储
- ⑤传输保护：提供了传输加密功能



4.1.1 身份鉴别 (Identification & Authentication)

DBMS提供的最外层安全保护措施，是最基本的系统安全手段

- 用户身份标识：用户名、UID（用户标识号）

- 用户名：手机号、邮箱、身份证号、姓名、代号等
- UID：系统唯一的内部编号

- 用户口令：一串符号

- 大小写字母和数字以及特殊符号组成的长度不小于8位（定期更换）



常用的用户身份鉴别方法：

1.静态口令鉴别

- 静态口令一般由用户自己设定，这些口令静态不变，方式简单，易被攻击，安全性较低。
- 可采用双因子鉴别，即口令+数字证书。
- 提供一套口令策略
- 用户身份鉴别可以重复多次

2.动态口令鉴别

- 口令是动态变化的，采用一次一密的方法
- 常用的方式：验证码和动态令牌方式
- 安全性相对高一些



3.生物特征鉴别

- 通过生物特征进行认证的技术，生物特征如指纹、虹膜和掌纹等
- 安全性较高

4.智能卡鉴别

- 智能卡是一种不可复制的硬件，内置集成电路的芯片，具有硬件加密功能
- 智能卡读取数据是静态的，通过内存扫描或网络监听等技术可能截取到用户的身份验证信息，存在安全隐患
- 实际应用中一般采用个人身份识别码（Personal Identification Number, PIN）和智能卡相结合方式



5.入侵检测

- 检测前，管理员按实际需求定义检测规则
- 检测中，实时进行入侵分析
- 发现入侵情况，实时进行处理
 - 通过邮件报警和断开会话连接、锁定用户进行处罚等



4.1.2 存取控制机制

- 通过用户和数据之间的“权限”匹配实现对用户访问数据的安全控制。
- 存取控制机制组成
 - 定义用户权限（权限即用户对某一数据对象的操作权力）
 - 合法权限检查
- 用户权限定义和合法权检查机制一起组成DBMS的存取控制子系统。



■ 常用存取控制方法

□ 自主存取控制 (Discretionary Access Control, 简称DAC)

➤ C2级

➤ 灵活

□ 强制存取控制 (Mandatory Access Control, 简称 MAC)

➤ B1级

➤ 严格



一、自主存取控制（DAC）

- 同一用户对于不同的数据对象有不同的存取权限
- 不同的用户对同一对象也有不同的权限
- 用户还可将其拥有的存取权限转授给其他用户

存取控制的表达

- 操作（数据对象）
- Select(TABLE student)
- CREATE(index)

- 
- 通过 SQL 的 GRANT 语句和 REVOKE 语句实现
 - 用户权限组成
 - 数据库对象
 - 操作类型
 - 定义用户存取权限：定义用户可以在哪些数据库对象上进行哪些类型的操作
 - 定义存取权限称为授权 (Authorization)

- 
- 定义存取权限 (对象权限): 用户
 - 检查存取权限 : DBMS
 - 授权粒度
 - 数据对象粒度: 数据库、表、属性列、行
 - 授权粒度越细, 授权子系统就越灵活, 能够提供的安全性就越完善。但另一方面, 因数据字典变大变复杂, 系统定义与检查权限的开销也会相应地增大。



1. SQL的授权功能

- GRANT语句的一般格式:

GRANT <权限>[,<权限>]...

ON <对象类型><对象名>[, <对象类型><对象名>]...

TO <用户>[,<用户>]...

[WITH GRANT OPTION];

- 将对指定操作对象的指定操作权限授予指定的用户。



(1) 用户的权限

- 基本表或视图的属主拥有对该表或视图的一切操作权限。
- 存取控制的对象类型不仅有数据本身，还有数据库和模式。

(2) 接受权限的用户

- 一个或多个具体用户
- PUBLIC (全体用户)



对象类型	对象	操 作 类 型
数据库 和 模式	数据库和模式	CREATE
	基本表	CREATE TABLE, ALTER TABLE
	视图	CREATE VIEW
	索引	CREATE INDEX
数据	基本表和视图	SELECT, INSERT, UPDATE, DELETE, REFERENCES, ALL PRIVILEGES
	属性列	SELECT, INSERT, UPDATE, REFERENCES, ALL PRIVILEGES

关系数据库系统中不同对象具有的主要存取权限

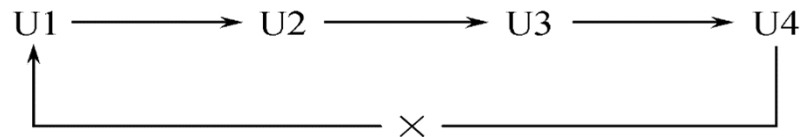
(3) WITH GRANT OPTION子句

- WITH GRANT OPTION子句:

- 指定: 可以再授予

- 没有指定: 不能传播

- 不允许循环授权, 即被授权者不能把权限再授回给授权者或其祖先。





MySQL创建用户

```
CREATE USER 'user_name'@'host' IDENTIFIED  
BY 'password';
```

- `user_name`: 要创建用户的名字。
- `host`: 表示要这个新创建的用户允许从哪台机登陆, 如果只允许从本机登陆, 则填'localhost', 如果允许从远程登陆, 则填'%'。
- `password`: 新创建用户的登陆数据库密码, 如果没密码可以不写。



例1 把查询Student表权限授给用户U1

```
GRANT SELECT  
ON TABLE Student  
TO U1;
```

SQLSever中语法:

Grant select on student to U1;

MySQL中对象类型是可选项:

object_type:

{ TABLE | FUNCTION | PROCEDURE }

例2 把对Student表和Course表的全部权限授予用户U2和U3

```
GRANT ALL [PRIVILIGES]  
ON TABLE Student, Course  
TO U2, U3;
```

注意MYSQL中，授权后必须FLUSH PRIVILEGES; 否则无法立即生效。



例3 把对表SC的查询权限授予所有用户

```
GRANT SELECT  
ON [TABLE] SC  
TO PUBLIC;
```

PUBLIC在MySQL 中不可用

例4 把查询Student表和修改学生姓名的权限授给用户U4

```
GRANT UPDATE(Sname), SELECT  
ON [TABLE] Student  
TO U4;
```




例5 把对表SC的INSERT权限授予U5用户，并允许他再将此
权限授予其他用户

```
GRANT INSERT  
ON [TABLE] SC  
TO U5  
WITH GRANT OPTION;
```



传播权限

执行例5后，U5不仅拥有了对表SC的INSERT权限，还可以传播此权限：

```
GRANT INSERT ON [TABLE] SC TO U6  
WITH GRANT OPTION;
```

同样，U6还可以将此权限授予U7：

```
GRANT INSERT ON [TABLE] SC TO U7;
```

但U7不能再传播此权限。



GRANT语句小结

- 一次向一个用户授权（例1）这是最简单的一种授权操作；
- 一次向多个用户授权（例2、例3）；
- 一次传播多个同类对象的权限(例2)；
- 一次可以完成对基本表、视图和属性列这些不同对象的授权（例4）；



2、SQL收回权限的功能

- REVOKE语句的一般格式为：

REVOKE <权限>[,<权限>]...

ON <对象类型><对象名>[, <对象类型><对象名>]...

FROM <用户>[,<用户>]... [CASCADE|RESTRICT];

- 功能：从指定用户那里收回对指定对象的指定权限



例6 把用户U4修改学生学号的权限收回

```
REVOKE UPDATE(Sno)  
ON TABLE Student  
FROM U4;
```

例7 收回所有用户对表SC的查询权限

```
REVOKE SELECT  
ON TABLE SC  
FROM PUBLIC;
```



例8 把用户U5对SC表的INSERT权限收回

```
REVOKE INSERT  
ON TABLE SC  
FROM U5;
```



■ 优点

- 能够通过授权机制有效地控制其他用户对敏感数据的存取

■ 缺点

- 可能存在数据的“无意泄露”
- 原因：这种机制仅仅通过对数据的存取权限来进行安全控制，而数据本身并无安全性标记。
- 解决：对系统控制下的所有主客体实施强制存取控制策略



二、强制存取控制方法 (MAC)

- 保证更高层次的安全性
- 用户不能**直接**感知或进行控制
- 适用于对数据有严格而固定密级分类的部门
 - 军事部门
 - 政府部门
- 每一个数据对象被标以一定的密级
- 每一个用户也被授予某一个级别的许可证
- 对于任意一个对象，只有具有合法许可证的用户才可以存取

- 
- 在MAC中，DBMS所管理的全部实体分为主体和客体两大类
 - 主体是系统中的活动实体
 - DBMS所管理的实际用户
 - 代表用户的各进程
 - 客体是系统中的被动实体，是受主体操纵的
 - 文件、基本表、索引、视图等



■ 敏感度标记 (Label)

- 对于主体和客体，DBMS为它们每个实例（值）指派一个敏感度标记 (label)
- 敏感度标记分成若干级别
 - 绝密 (Top Secret)
 - 机密 (Secret)
 - 可信 (Confidential)
 - 公开 (Public)
- 主体的敏感度标记称为**许可证级别** (Clearance Level)
- 客体的敏感度标记称为**密级** (Classification Level)



不允许低安全等级的用户读取高安全等级的信息，
不允许高敏感度的信息写入低敏感度的区域，
禁止信息从高级别流向低级别。

■ 强制存取控制规则（Bell-LaPadula模型，不上读，不下写）

- (1) 仅当主体的许可证级别**大于或等于**客体的密级时，该主体才能**读**取相应的客体
- (2) 仅当主体的许可证级别**小于或等于**客体的密级时，该主体才能**写**相应的客体

除此之外，还有Biba模型，Chinese Wall模型等



规则的共同点:

- 禁止了拥有高许可证级别的主体更新低密级的数据对象
- 防止数据的密级从高流到低，造成数据泄漏

- 
- MAC是对数据本身进行密级标记，无论数据如何复制，标记与数据是一个不可分的整体，只有符合密级标记要求的用户才可以操纵数据。
 - 实现MAC时要**首先**实现DAC
 - 原因：较高安全性级别提供的**安全保护要包含较低级别的所有保护**
 - DAC与MAC共同构成**DBMS**的安全机制

- 
- MAC规则告诉我们，用户只能读密级比自己低的数据，但是可以写密级比自己高的数据，这意味着写进去的数据自己将看不见，这合理吗？
 - MAC实质：单向信息流，信息只能低等级向高等级流动，相反的话，就造成信息的泄露。
 - MAC难点：如何确定主客体的密级？数据的分类分级也是当下政务信息共享中的难题之一。



4.1 数据库安全性控制

4.1.1 用户标识与鉴别

4.1.2 存取控制

4.1.3 数据库角色

- 
- 数据库角色 (ROLE) : 被命名的一组与数据库操作相关的权限
 - 角色是权限的集合
 - 可以为一组具有相同权限的用户创建一个角色
 - 简化授权的过程
 - MySQL8.0中新增



(一) 角色的创建

CREATE ROLE <角色名>

(二) 给角色授权

GRANT <权限> [, <权限>] ...

ON <对象类型>对象名

TO <角色> [, <角色>] ...



提问：角色是用户还是权限？

(1) 角色可以**看做**是一类特殊的用户，从授权语句就可以看出来：

GRANT SELECT, UPDATE, INSERT ON TABLE Student TO R1;

(2) 角色又**不是**一般的用户，角色**是权限的集合**。因此，角色也可以授权给其他用户或者角色。

GRANT R1 TO U2;

(3) 角色又不是一般的权限，不能用WITH GRANT OPTION来转授权，需要用特殊的语句来转授权（WITH ADMIN OPTION）



(三) 将一个角色授予其他的角色或用户

GRANT <角色1> [, <角色2>] ...

TO <角色3> [, <用户1>] ...

[WITH ADMIN OPTION] -- 传播授权

(四) 角色权限的收回

REVOKE <权限> [, <权限>] ...

ON <对象类型> <对象名>

FROM <角色> [, <角色>] ...



例9 通过角色来实现将一组权限授予一组用户。

1. 首先，创建一个角色ml_role

```
create role 'ml_role'; /*创建角色*/
```

2. 然后，使用Grant语句，使角色ml_role
拥有Student表的Select、Insert和Delete权限。

```
Grant select, insert, delete On ml_m.student to 'ml_role'@'%'; /*为角色授权*/
```

- 
3. 将这个角色授予Db_test和Db_user2。使他们具有角色ml_role所包含的全部权限。

```
create user Db_test identified by 'Zn123456'; /*创建用户*/  
create user 'Db_user2'@'%' identified by 'Zn123456';  
Grant 'ml_role'@'%' to Db_test,'Db_user2'@'%'; /*把角色分配给用户*/
```

4. 可以一次性地回收Db_test的这三个权限

```
Revoke 'ml_role'@'%' from Db_test;  
show grants for Db_test;
```

5. **激活role**：默认情况下，Role 在分配给用户后 不会自动生效，需手动激活。



激活方式有两种：

(1) 临时激活（当前会话有效）

`SET ROLE 'role_name';` -- 激活单个 Role

`SET ROLE ALL;` -- 激活所有分配给用户的 Role

(2) 永久激活（用户登录时自动激活）

-- 方法1：修改用户默认 Role

`ALTER USER 'user_name'@'host' DEFAULT ROLE 'role_name';`

-- 方法2：设置全局变量（MySQL 8.0.2+）

`SET GLOBAL activate_all_roles_on_login = ON;` -- 所有用户登录时自动激活 Role



例9 通过角色来实现将一组权限授予一组用户。

1. 首先，创建一个角色R1

Create role R1;

2. 然后，使用Grant语句，使角色R1拥有Student表的Select、Insert和Delete权限。

Grant Select, Insert ,Delete On Student to R1;

- 
3. 将这个角色授予王平，张明，赵玲。使他们具有角色R1所包含的全部权限。

Grant R1 to 王平, 张明, 赵玲;

4. 可以一次性地回收王平的这三个权限

Revoke R1 from 王平;



第4章 数据库安全性

4.1 数据库安全性控制

4.2 视图机制 (略)

4.3 审计

4.4 数据加密

4.5 其他安全性保护 (自学)



4.2 视图机制

- 把要保密的数据对无权存取这些数据的用户隐藏起来，对数据提供一定程度的安全保护
- 视图机制配合授权机制，间接地实现支持存取谓词的用户权限定义



例10 建立计算机系学生的视图，把对该视图的
SELECT权限授予王平，把该视图上的所有操作权
限授予张明。

先建立计算机系学生的视图CS_Student

```
CREATE VIEW CS_Student
AS
  SELECT *
  FROM Student
  WHERE Sdept='CS';
```



在视图上进一步定义存取权限

```
GRANT SELECT  
ON CS_Student  
TO 王平;
```

/*王平老师只能检索计算机系学生的信息*/

```
GRANT ALL PRIVILIGES  
ON CS_Student  
TO 张明;
```

/*张明具有检索和增删改该视图的所有权限*/



4.3 审计

■ 审计

- 启用一个专用的审计日志 (Audit Log)

把用户对数据库的所有操作自动记录放入审计日志

- 审计员利用审计日志

监控数据库中的各种行为，找出非法存取数据的人、时间和内容

- C2以上安全级别的数据库管理系统必须具有审计功能



■ 审计功能的可选性

- 审计很费时间和空间
- DBA可以根据应用对安全性的要求，灵活地打开或关闭审计功能
- 审计功能主要用于安全性要求较高的部门



1.审计事件

☐ 服务器事件

- 审计数据库服务器发生的事件

☐ 系统权限

- 对系统拥有的结构或模式对象进行操作的审计
- 要求该操作的权限是通过系统权限获得的

☐ 语句事件

- 对SQL语句，如DDL、DML及DCL语句的审计

☐ 模式对象事件

- 对特定模式对象上进行的SELECT或DML操作的审计



2.审计功能

- 基本功能：提供多种审计查阅方式
- 多套审计规则：一般在初始化设定
- 提供审计分析和报表功能
- 审计日志管理功能
 - 防止审计员误删审计记录，审计日志必须先转储后删除
 - 对转储的审计记录文件提供完整性和保密性保护
 - 只允许审计员查阅和转储审计记录，不允许任何用户新增和修改审计记录等
- 提供查询审计设置及审计记录信息的专门视图



3. AUDIT语句和NOAUDIT语句

- AUDIT语句：设置审计功能
- NOAUDIT语句：取消审计功能



■ 用户级审计

- 任何用户可设置的审计
- 用户针对自己创建的数据库表和视图进行审计

■ 系统级审计

- 只能由数据库管理员设置
- 监测成功或失败的登录要求、监测授权和收回操作以及其他数据库级权限下的操作



例11 对修改SC表结构或修改SC表数据的操作进行审计

```
AUDIT ALTER, UPDATE  
ON SC;
```

例12 取消对SC表的一切审计

```
NOAUDIT ALTER, UPDATE  
ON SC;
```

注：MySQL8.0不支持此语句，可使用日志方式实现。



4.4 数据加密

- 防止数据库中数据在存储和传输中失密的有效手段
- 加密的基本思想
 - 根据一定的算法将原始数据—明文 (plain text) 经过一系列复杂计算, 变换为不可直接识别的格式—密文 (cipher text)
- 分类
 - 存储加密和传输加密



1.存储加密

■ 透明存储加密

- 内核级加密保护方式，对用户完全透明
- 数据在写到磁盘时对数据进行加密，授权用户读取数据时再对其进行解密
- 数据库的应用程序不需要做任何修改，只需在创建表语句中说明需加密的字段即可

内核级加密方法: 性能较好, 安全性较高

■ 非透明存储加密

- 通过多个加密函数实现



2.传输加密

□ 链路加密

- 在链路层进行加密
- 传输信息由报头和报文两部分组成
- 报文和报头均加密

□ 端到端加密

- 在发送端加密，接收端解密
- 只加密报文,不加密报头
- 发送端和接收端需要密码设备，中间节点不需要密码设备
- 所需密码设备相对较少，容易被非法监听者获取敏感信息



❖ 数据加密算法

- 对称算法加密: AES(Advanced Encryption Standard)算法
- 非对称加密算法: RSA算法
- DBMS自己设计和采用特定的加密算法

❖ 数据加密优缺点

- 数据安全性进一步提高
- 增加查询处理复杂性, 查询效率受到影响

- DBMS中的数据加密

- 有些数据库产品提供了数据加密例行程序
- 有些数据库产品本身未提供加密程序，但提供了接口

- 数据加密功能通常也作为可选特征，允许用户自由选择

- 数据加密与解密是比较费时的操作
- 数据加密与解密程序会占用大量系统资源
- 应该只对高度机密的数据加密



4.6 其他安全性保护

1. 推理控制

- 处理强制存取控制未解决的问题
- 避免用户利用能够访问的数据推知更高密级的数据
- 常用方法
 - 基于函数依赖的推理控制
 - 基于敏感关联的推理控制

2. 隐蔽信道

- 处理强制存取控制未解决的问题



3.数据隐私

- 控制不愿他人知道或他人不便知道的个人数据的能力
- 范围很广：数据收集、数据存储、数据处理和数据发布等各个阶段

4. “三权分立”的安全管理机制

- 解决数据库管理员权限过于集中的问题，遵照 GB/T20273-2019，引进“三权分立”的安全管理机制



本章作业

- 思维导图总结本章内容。
- P148-1, 3, 4, 6, 7（去掉③⑦小题）, 10.